

中級編 マルチGPUプログラミング (OpenACC, C/C++)

次世代HPC・AI研究開発支援センター(HAIRDESC)

資料本編前に

- 本資料は, 単体GPU化済みのコードをMPIを用いてマルチGPU化したい方向け
 - 単体GPU化手法については, 別資料を参照のこと:
『初級編 単一GPUプログラミング』のOpenACC版
- プログラミング言語としてはC/C++を用いる

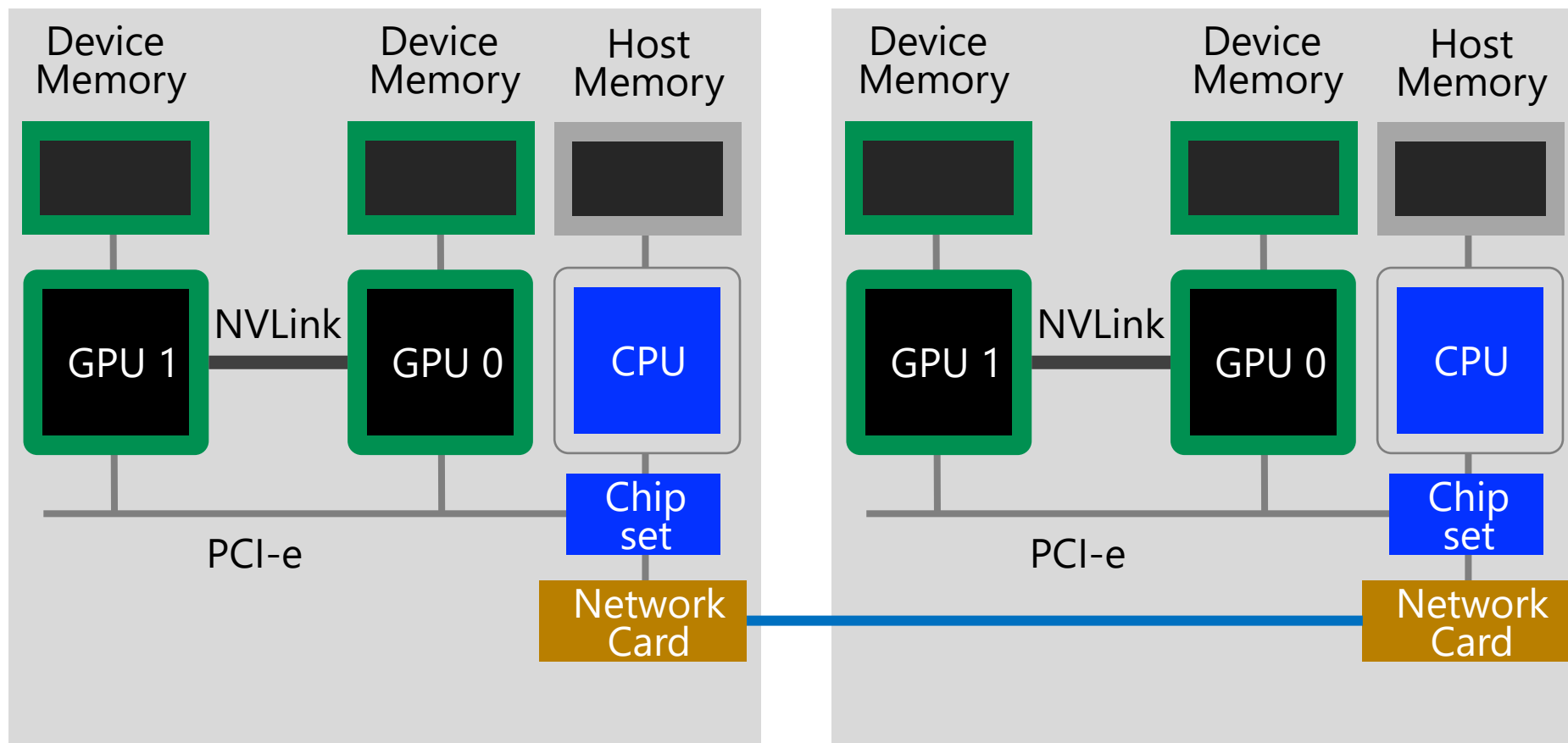
Section 1.

複数GPUの使用イメージ

マルチGPUコンピューティング

- 複数GPUを使う目的は？
 - 1個のGPUに搭載されたメモリではカバーしきれない大きい問題を解きたい
 - 1個のGPUで計算するよりも高速に計算したい
- 複数GPUを使う方法は？
 - 推奨: 複数GPUをMPIで並列化
 - 最大の難点は環境構築の難しさ ← GPUスパコンでは, このあたりのお膳立てが整っている
 - MPIであれば, 複数ノードに搭載された複数GPUの活用も容易
 - 推奨: MPIプロセスあたり1 GPUとして利用するのが最も簡単
 - MPIプロセスあたりに複数GPUを割り当てることも可能(`acc_set_device_num` (OpenACC), `cudaSetDevice` (CUDA) などでGPUを切り替えながら計算することになる)
 - 非推奨: 複数GPUをOpenMP(for マルチコアCPU)で並列化
 - できなくはない, 環境構築は容易
 - OpenMPを用いて複数GPU対応コードに書き換えても, 複数ノードに拡張できない

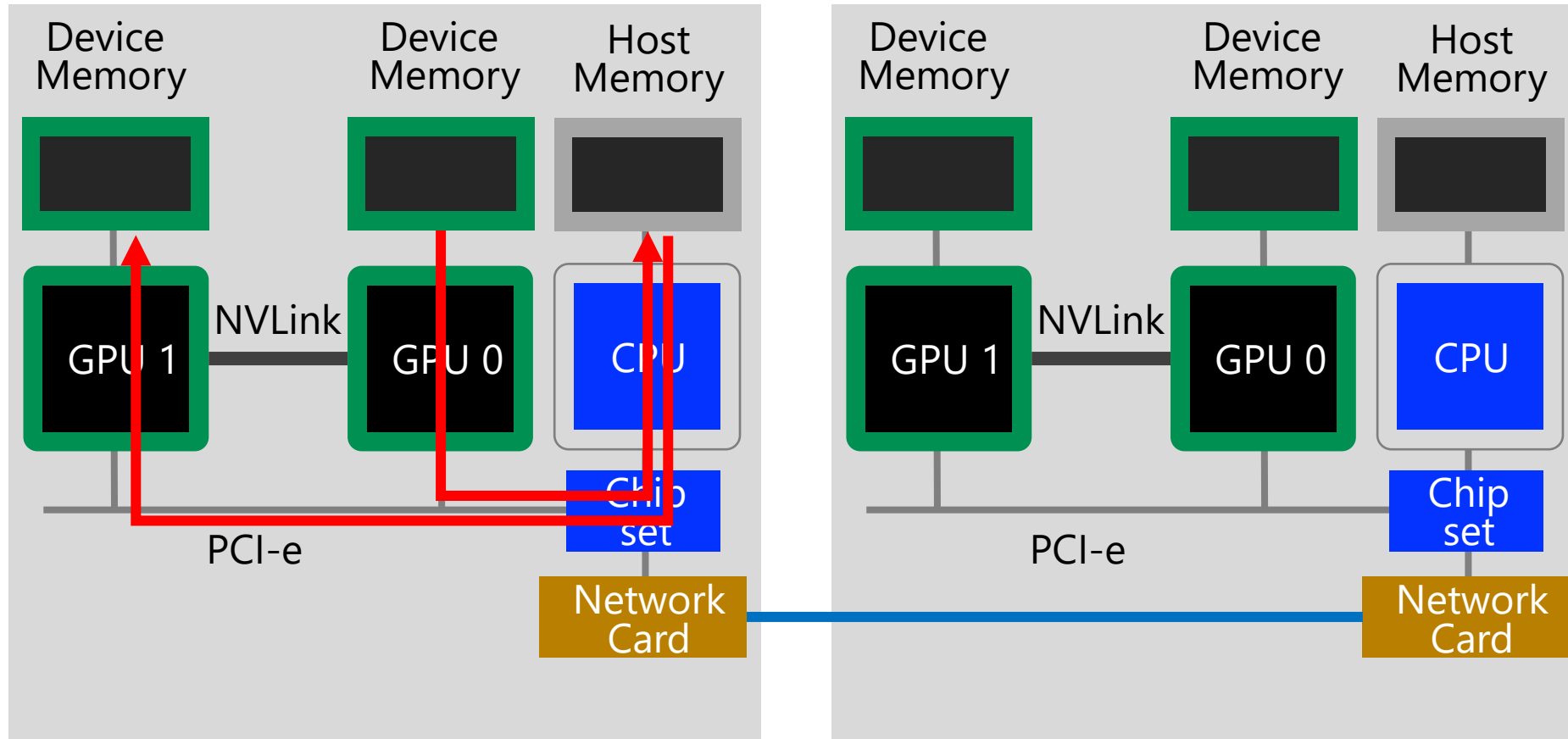
GPUx2 の計算ノード模式図



- GPU-GPU通信にはいくつかの種類がある

ノード内GPU間通信(1)

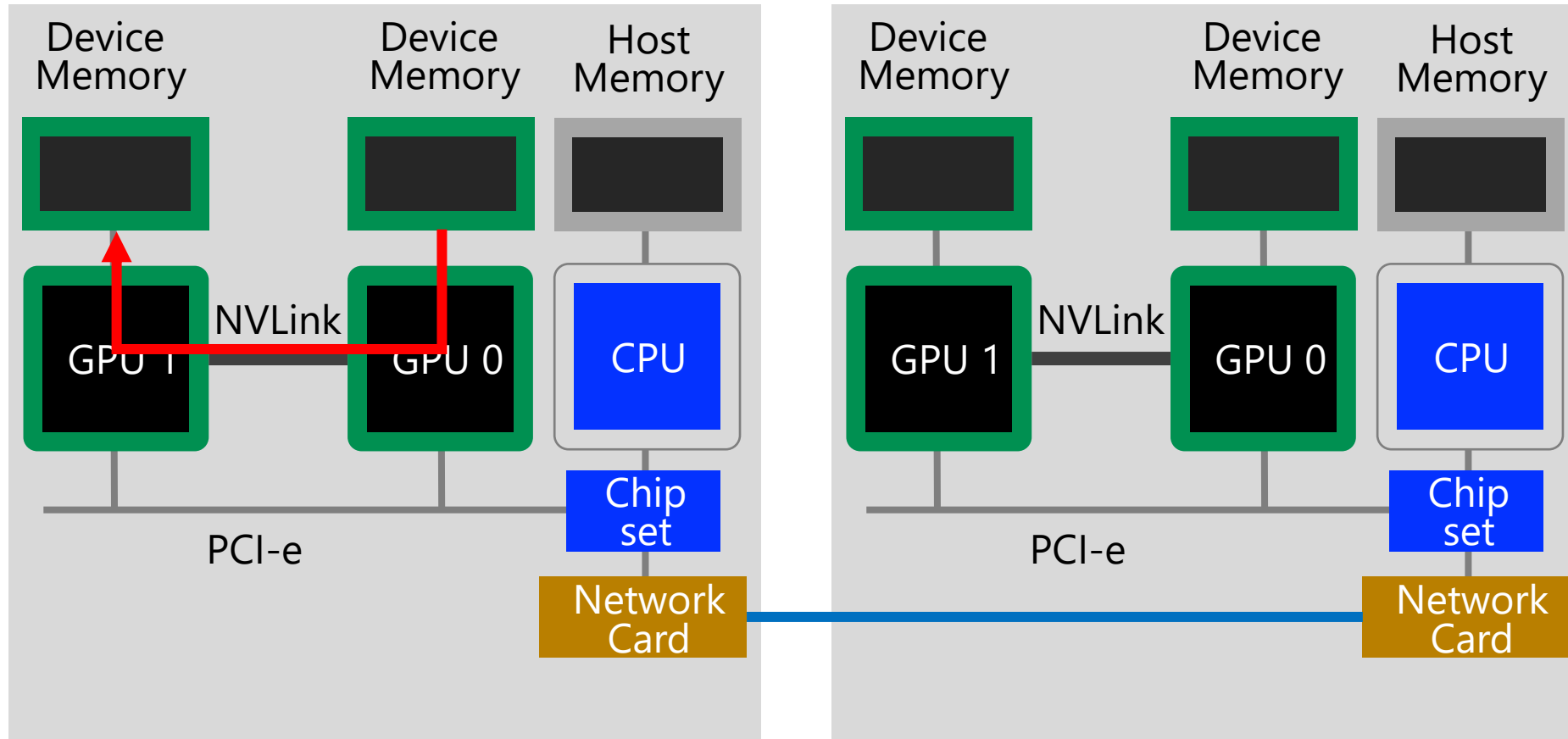
- ホストメモリ経由のノード内GPU間通信
 - ✓ デバイスメモリとホストメモリの間の PCI expressを通る



ノード内GPU間通信(2)

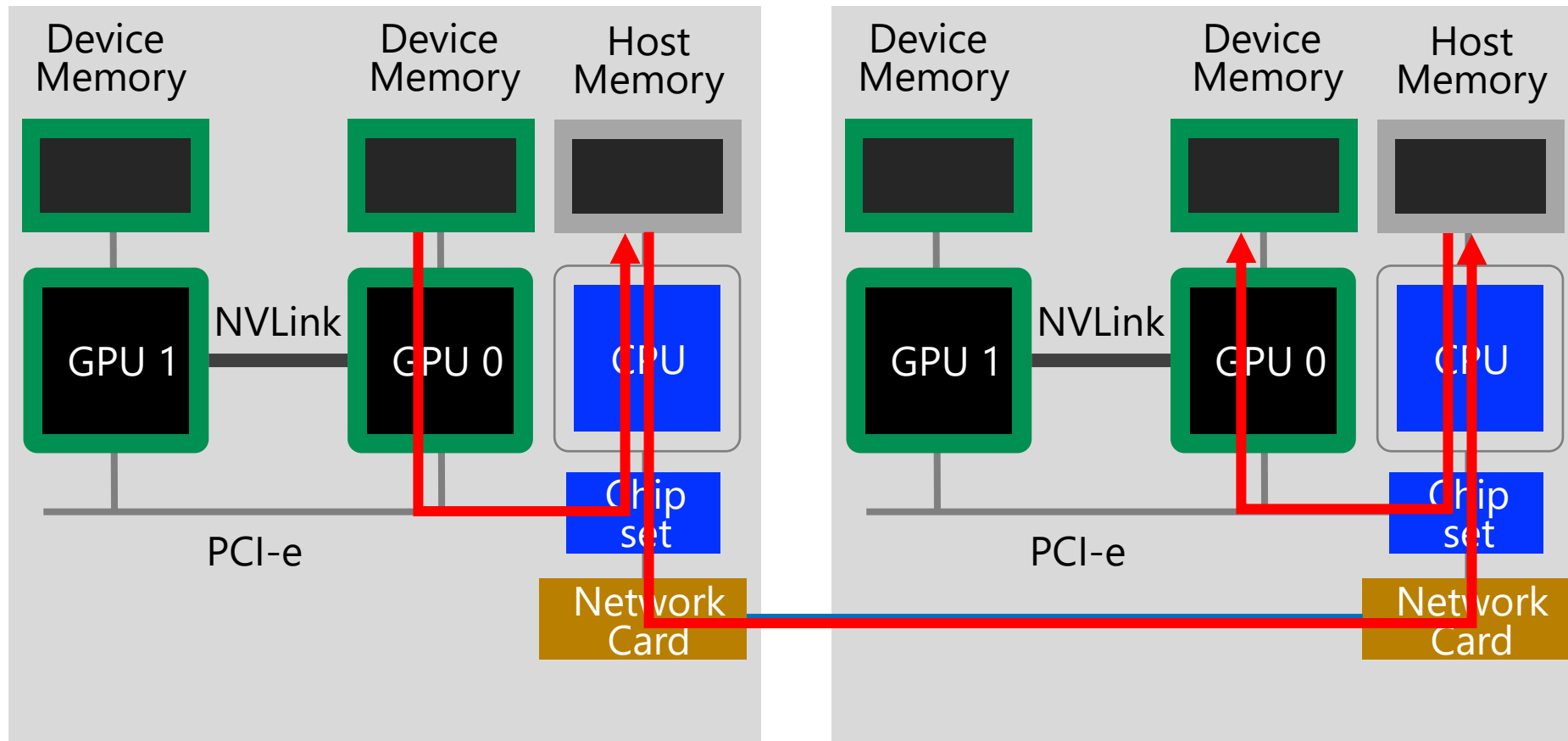
- NVLink経由のGPU間通信

- ✓ CUDA Inter Process Communication (IPC) を利用し、同じノード内にあるプロセスではホストを経由せずGPU間で直接通信できる
- ✓ 最近のサーバではGPU間をつなぐNVLink経由で高速通信する



ノード間のGPU間通信(1)

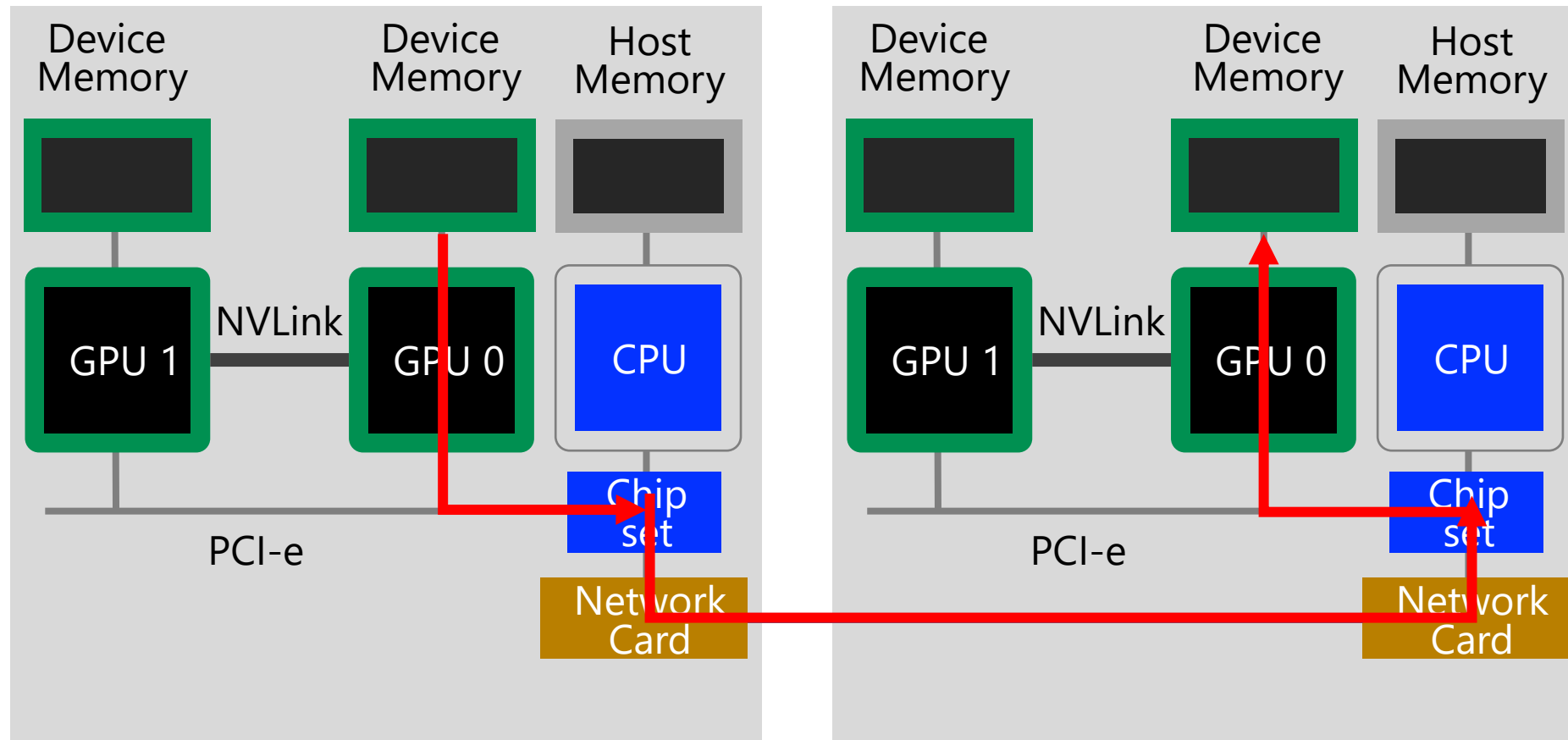
- ホストメモリ経由のノード間GPU間通信
 - ✓ あるノードのGPU0から別ノードのGPU0へ転送する際に, それぞれのホストメモリを経由して転送する



ノード間のGPU間通信(2)

- GPUDirectRDMAによるノード間GPU間通信

- ✓ ホストメモリを経由することなく, GPUとInfiniBand (Network Card) 間で直接データ転送 (RDMA) をすることにより異なるノードのGPU間で高速通信する



Section 2.

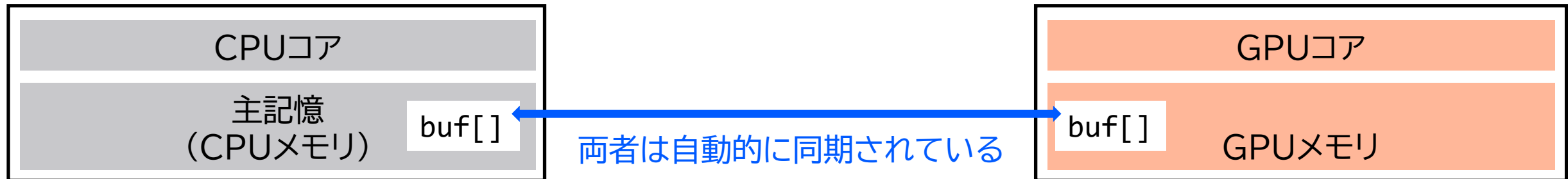
マルチGPU化方針・実装方法

具体的な実装方法

- 大まかには3通りの実装方法がある
- ① Managed/Unified Memory を用いたGPU化を行っている場合
 - 汎用CPU向けのMPI並列化の実装と何も変わらない(変えなくて良い)
 - ユーザーは実際のデータがCPU側にあるかGPU側にあるか意識せずに実装し、データ管理についてはバックエンドにお任せしている
- GPUに置くデータとCPUに置くデータをユーザーが管理している場合
 - ② GPU-Aware MPI(後述)を用い, GPU上のアドレスをMPI関数に直接指定
 - MPI関数に対して, GPU上のアドレスを指定したことを教えてあげる
 - GPUスパコン上で一番高い性能が得られる方法であり, 実装コストも低い
 - ③ GPU上のデータを送受信する際に, ユーザー自身でCPU-GPU間データ転送を行った上で, CPU上のアドレスをMPI関数に指定
 - CPU-GPU間のデータ転送を自分で記述する必要があるが, どの環境でも動作する方法

CPU・GPU両側のメモリが参照できる場合

- Managed/Unified Memoryを使っている場合
- MPI関数関連の実装については、一切変更する必要がない
 - データ管理については、全てバックエンドにおまかせしている状態
 - MPI通信のためにCPU-GPU間データ転送が必要であったとしても、自動的に処理してくれる
- 不必要なCPU-GPU間データ転送が生じることもある点に注意
 - GPU上のデータをCPU側にコピーした上でCPU-CPU間でのMPI通信が発行される場合もある



GPU-Aware MPI

- 従来の(あるいは通常の)MPI
 - MPIの受信領域や送信領域にホストメモリ上のアドレスのみ指定可能
 - デバイスメモリのデータを転送する際には, 一度ホストメモリへコピーが必要
- GPU-Aware MPI
 - MPIの受信領域や送信領域にデバイスメモリ上のアドレスも指定可能
 - MPIライブラリ内部ではGPUDirectなどが利用され, MPIライブラリを利用するだけでGPU 間的高速通信技術を利用することができる
 - →実装がシンプルになり, さらに通信性能も上がる
 - OpenACC, CUDA などから利用できる
 - Managed memory との相性は悪い(GPU上のアドレスと特定できないため)

GPU-Aware MPIを用いた実装

- MPIの受信領域や送信領域にデバイスメモリ上のアドレスを指定可能

- デバイスポインタであるということを教えてあげるだけで良い

- CUDAであれば `cudaMalloc()` で確保していればそのままOK

- OpenACC ではホスト・デバイスメモリ上の配列名が共通なので, 追加指示が必要

- `#pragma acc host_data use_device(...)` を追加

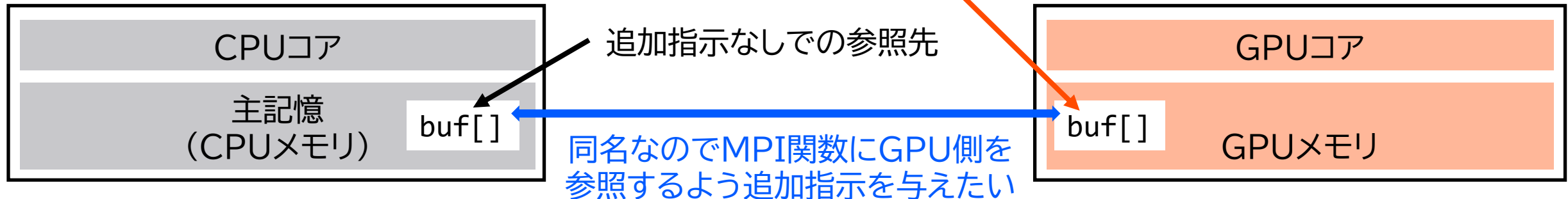
```
#pragma acc host_data use_device(recv_buf)
```

```
MPI_Irecv(recv_buf, recv_count, MPI_DOUBLE, recv_from, tag, MPI_COMM_WORLD, &recv_req);
```

```
#pragma acc host_data use_device(send_buf)
```

```
MPI_Isend(send_buf, send_count, MPI_DOUBLE, send_to, tag, MPI_COMM_WORLD, &send_req);
```

`#pragma acc host_data use_device(buf)` 付与時の参照先



自分でCPU-GPU間のデータ転送を記述

- GPU-Aware MPI未提供の環境でも動作する方法
 - (ただしGPUスパコン上では, GPU-Aware MPIを使う方が楽で高性能)
 - 実装方法は単体GPUのときのCPU-GPU間データ転送を追加するだけ
 - MPI通信については全てCPUにお任せし, 全データをCPU経由で送受信する方式

```
#pragma acc data create(a[0:n], b[0:n])
{
  #pragma acc kernels
  #pragma acc loop independent
  for (unsigned int i=0; i<n; i++) {
    a[i] = 3.0 * rank * ny;
    b[i] = 0.0;
  }
  const int dst_rank = (rank + 1) % nprocs;
  const int tag      = 0;
  if (rank == 0) {
    MPI_Status status;
    MPI_Recv(b, w * nx, MPI_FLOAT, dst_rank, tag, MPI_COMM_WORLD, &status);
    #pragma acc update device(b[0:w * nx])
  } else {
    #pragma acc update host(a[0:w * nx])
    MPI_Send(a, w * nx, MPI_FLOAT, dst_rank, tag, MPI_COMM_WORLD);
  }
  // 以降の実装
}
```

MPI(H)
H→D
D→H
MPI(H)

MPI通信前後に挿入

Section 3.

コンパイル・実行方法

MPI+OpenACCコードのコンパイル方法

- 単体OpenACCコードのコンパイラをMPIコンパイラに変更
 - (下記はC++向け. C言語の場合はmpiccを使用)
- Unified Memory 使用時のコンパイル方法
 - `$ mpic++ -acc=gpu -gpu=cc90,mem:unified:nomanagedalloc ¥ -Minfo=accel,opt`
 - リンク時にも `-acc=gpu -gpu=mem:unified:nomanagedalloc` を指定
 - `-gpu=cc90`は, 用いるGPUの世代(90はHopper(H100, GH200など))を指定
- Managed Memory 使用時のコンパイル方法
 - `$ mpic++ -acc=gpu -gpu=cc90,mem:managed -Minfo=accel,opt`
 - リンク時にも `-acc=gpu -gpu=mem:managed` を指定
- Separate 使用時のコンパイル方法(OpenACCのデフォルト)
 - `$ mpic++ -acc=gpu -gpu=cc90 -Minfo=accel,opt`
 - リンク時にも `-acc=gpu` を指定

環境変数を用いたGPUマッピング

- 主にノードあたりに複数GPUが搭載されているシステム向けの情報
 - Miyabi-G のようなノードあたり1 GPUのシステムでは, この内容は不要
- `acc_set_device_num()`, `cudaSetDevice()`などを使わずに, 各MPIプロセスがどのGPUを用いるかをプログラム外から制御できる
 - NVIDIA製GPUの場合には, `CUDA_VISIBLE_DEVICES`
- 例えばMPIプロセスあたりのGPU数を1とする(これがおすすめ)場合には,
 - `$ mpiexec -n 4 ./wrapper.sh ./a.out`
 - `wrapper.sh` の中身(`chmod +x` をお忘れなく):

```
#!/usr/bin/env bash
local_rank=${OMPI_COMM_WORLD_LOCAL_RANK:=${MV2_COMM_WORLD_LOCAL_RANK}}
GPU_ID=${local_rank}
export CUDA_VISIBLE_DEVICES=${GPU_ID}
$*
```

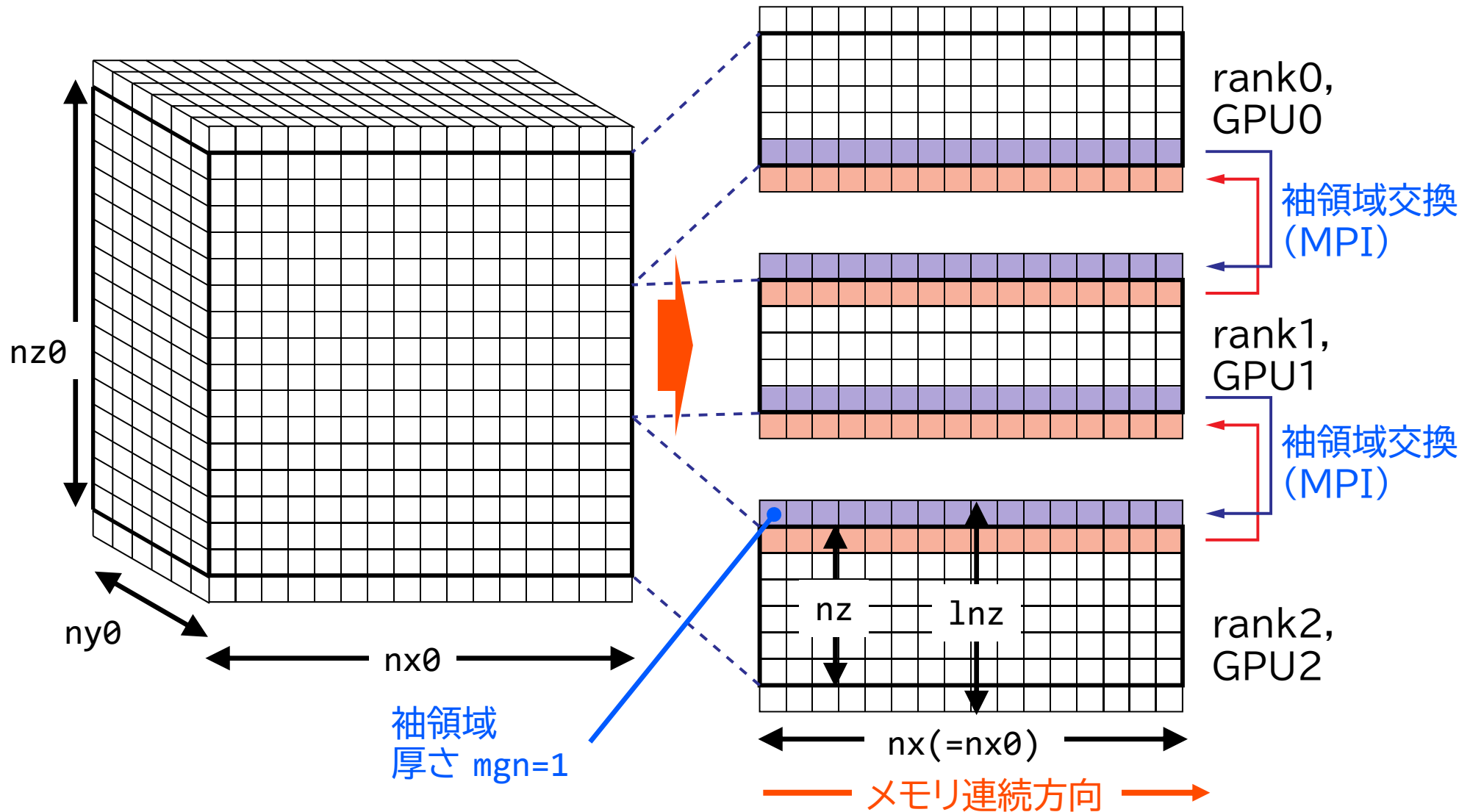
- Open MPI/MVAPICH2に対応したスクリプト例
- `*_LOCAL_RANK` は, ノード内のMPIランクを示す変数(0, 1, ...)
- 注:MIGやMPSを使う際には`CUDA_VISIBLE_DEVICES`にはUUIDを指定することになる

Section 4.

マルチGPU化例

拡散方程式コードの領域分割方針

- 領域分割し、複数GPUが各領域を担当
- 計算領域は $nx0 * ny0 * nz0$
- z方向に1次元分割
 - 袖領域がメモリ空間上で連続
- 上下に隣接するGPU間でのMPI通信が必要



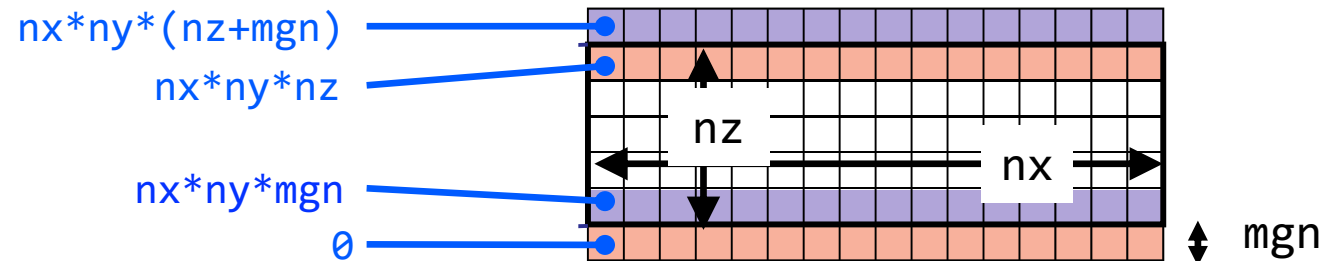
CPU版コードにおけるMPI実装

- diffusion3d 関数(時間発展計算)の実行前に
袖領域のデータ交換のためにMPI_Send/MPI_Recv が実行される
 - diffusion3d関数のGPU化については、『単一GPUプログラミング』を参照
 - Unified/Managed Memory使用時には、この時点でマルチGPU化も完了

```
for (; icnt < nt && time + 0.5 * dt < 0.1; icnt++) {
    const int tag = 0;
    MPI_Status status;
    MPI_Send(&f[nx * ny * nz], nx * ny, MPI_FLOAT, rank_up, tag, MPI_COMM_WORLD);
    MPI_Recv(&f[0], nx * ny, MPI_FLOAT, rank_down, tag, MPI_COMM_WORLD, &status);

    MPI_Send(&f[nx * ny * mgn], nx * ny, MPI_FLOAT, rank_down, tag, MPI_COMM_WORLD);
    MPI_Recv(&f[nx * ny * (nz + mgn)], nx * ny, MPI_FLOAT, rank_up, tag, MPI_COMM_WORLD, &status);

    diffusion3d(nprocs, rank, nx, ny, nz, mgn, dx, dy, dz, dt, kappa, f, fn);
    swap(&f, &fn);
    time += dt;
}
```



GPU-Aware MPI版実装

- diffusion3d 関数(時間発展計算)の実行前に
袖領域のデータ交換のためにMPI_Send/MPI_Recv が実行される
 - diffucion3d 関数は, OpenACCによる単体GPU化がなされている

```
#pragma acc data copy(f[0 : Ln], fn[0 : Ln])
{
    for (; icnt < nt && time + 0.5 * dt < 0.1; icnt++) {
        const int tag = 0;
        MPI_Status status;

        #pragma acc host_data use_device(f)
        {
            MPI_Send(&f[nx * ny * nz], nx * ny, MPI_FLOAT, rank_up, tag, MPI_COMM_WORLD);
            MPI_Recv(&f[0], nx * ny, MPI_FLOAT, rank_down, tag, MPI_COMM_WORLD, &status);

            MPI_Send(&f[nx * ny * mgn], nx * ny, MPI_FLOAT, rank_down, tag, MPI_COMM_WORLD);
            MPI_Recv(&f[nx * ny * (nz + mgn)], nx * ny, MPI_FLOAT, rank_up, tag, MPI_COMM_WORLD, &status);
        }

        diffusion3d(nprocs, rank, nx, ny, nz, mgn, dx, dy, dz, dt, kappa, f, fn);
        swap(&f, &fn);
        time += dt;
    }
}
```

GPU上の配列fをMPI関数に渡すことを指示

Section 5.

関連情報

資料提供者一覧

- 五十音順, 敬称略
- 下川辺 隆史(東京大学 情報基盤センター)
 - Section 4
- 星野 哲也(名古屋大学 情報基盤センター)
 - Sections 1, 4
- 三木 洋平(東京大学 情報基盤センター)
 - Sections 1, 2, 3, 4

資料作成者一覧

- 五十音順, 敬称略
- 三木 洋平(東京大学 情報基盤センター)
 - 2026/3/25版